ML Project Deep Dive Pre-Read: Dynamic Pricing Engine (DPE)

Rudraksh Syal, Senior Data Scientist, Vista

All figures are from a full rebuild on anonymised, representative data. Specific numbers should not be attributed to any company's proprietary data.

---

1 — Problem Statement

- **Company / marketplace context:** Vista Tech (dba FLYXO) operates in private aviation with a hybrid model: Program customers subscribe for priority fleet access, and whatever capacity is left is sold to on-demand customers on a per-flight basis at a quoted yield ($ per EFT hour, where EFT = Estimated Flight Time).
- **Prior system:** Pricing was mostly based on static MSRP-style bands. These bands were informed by route, aircraft, seasonality, and booking-curve signals, but they still did not react well to the actual live supply-demand state.
- **Why this mattered:** The inventory is highly perishable. If a tail is unused on Tuesday, we cannot sell that same Tuesday capacity on Wednesday. At the same time, overcommitting can force off-fleet sourcing, which can be break-even or loss-making. So the business risk existed on both sides: pricing too low when supply was scarce, or pricing too high / too rigid when capacity was available.
- **My role:** I led the project as the Senior Data Scientist, with two junior data scientists supporting parts of the analysis. I owned the end-to-end DS work: problem framing, modeling, validation, stakeholder alignment, revenue leadership readouts, documentation, and production deployment through Snowflake / SageMaker / Airflow.

---

2 — Solutions Explored

- **Option 1 — Improve the existing MSRP rules:**
  We considered adding more seasonality and days-to-departure adjustments to the rule system. This was simple and familiar, but it still would not react to sudden supply changes and did not create a real feedback loop between supply, demand, and price.
- **Option 2 — Train an end-to-end ML model to predict bookings from yield:**
  This was attractive at first, but the historical prices were mostly generated by the existing MSRP formula. That meant yield was heavily confounded with route, aircraft, seasonality, and human pricing behavior. A black-box model also would have been hard for the pricing team to trust, especially when it was replacing a workflow they already understood.
- **Chosen approach — Supply-Demand equilibrium solver:**
  I separated the problem into two pieces: forecast available OD supply and model OD demand as a function of price. Then I solved for the yield Y* where expected demand equals available OD supply. This approach was more interpretable, easier to validate in pieces, and better aligned with how pricing stakeholders thought about the business.

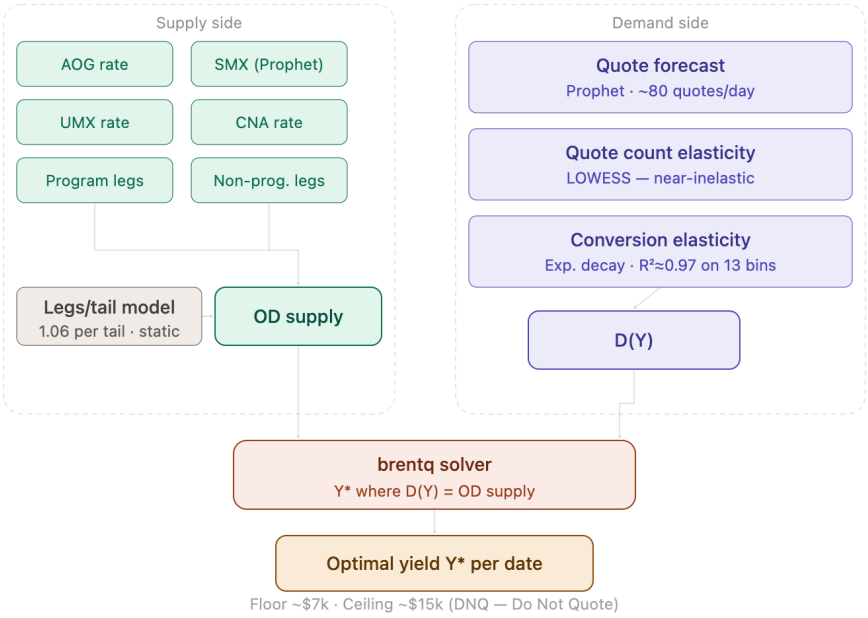


Figure 1 — DPE system architecture: Supply sub-models (teal) + demand components (purple) → solver (coral) → optimal yield Y per departure date.*

---

3 — Technical Considerations

- **Supply model:**
  OD Supply = (delivered tails × 1.06) − Program legs − Non-Program legs − Σ(OD legs × (1 − cancellation rate by callout bucket)).

  The supply side used six components:

  - ☐ AOG and UMX (seasonal mean models)
  - ☐ SMX (Prophet model, about 18% improvement versus baseline)
  - ☐ CNA (constant rate, since the data did not show meaningful seasonality)
  - ☐ Program legs (Prophet model)
  - ☐ Non-Program legs (seasonal mean model)
- **Demand model:**
  D(Y) = quote forecast × quote elasticity(Y) × conversion elasticity(Y)

  One booking was treated as almost one OD leg at the fleet-day level, which was consistent with the historical data. Quote volume was forecasted with Prophet. Quote-count elasticity was modeled with LOWESS and was close to flat, which suggested that private aviation shoppers tend to inquire even when prices move. Conversion elasticity was modeled with an exponential decay curve: $a \cdot \exp(-b \cdot Y)$.
- **Important caveat on R^2:**
  The conversion curve had $R^2 \approx 0.97$, but this is a curve-fit metric across 13 aggregated yield bins using roughly 50k quotes. I treated it as evidence that the exponential shape fit the observed data well — not as per-quote prediction accuracy and not as a fully causal estimate.

- **Solver:**
  I used `brentq` to solve for the market-clearing yield on a bounded interval. The lower bound was around $7k, which represented the operational floor. The upper bound was around $15k, above which the recommendation became DNQ (Do Not Quote). Brent's method was a good fit because it is stable, bounded, and does not require tuning hyperparameters.
- **Stack:**
  Python, Pandas, Polars, Prophet, scipy, MLflow. Production deployment used Snowflake, AWS SageMaker, and Apache Airflow with a daily batch refresh.

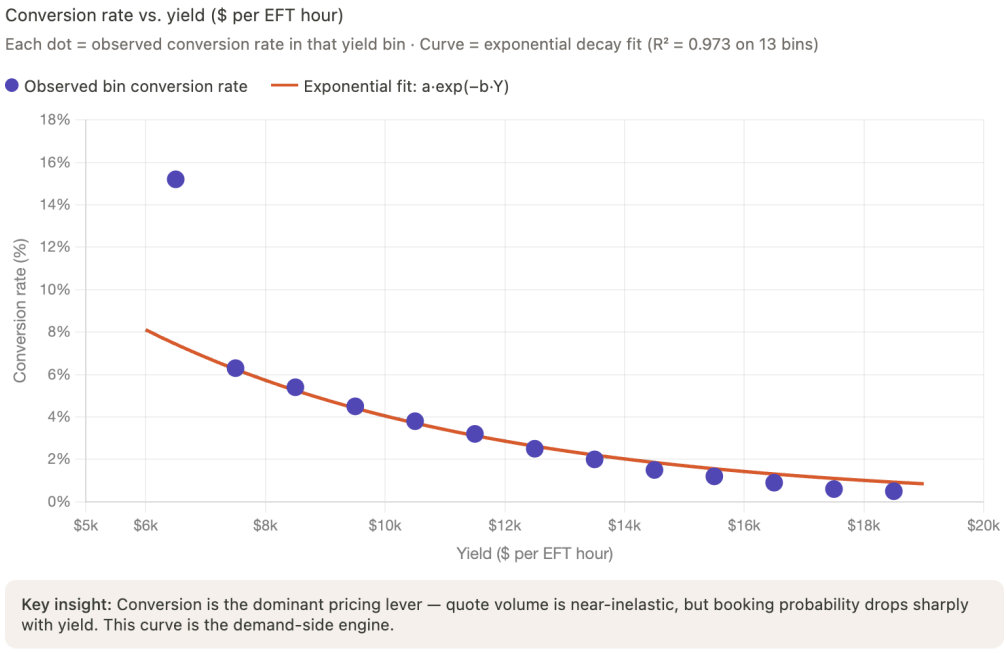


Figure 2 — Conversion elasticity: exponential decay fit

---

4 — Measuring Success

January 2026 validation (directional and representative figures from anonymised rebuild):

Metric	Baseline	Result
OD supply visibility	No daily forecast	~1.5 legs/day mean; 16 of 31 Jan days = zero supply (DNQ) — previously undetected
Zero-supply day handling	Quotes issued on zero-supply days → off-fleet risk	16 days identified as DNQ pre-hoc; off-fleet exposure avoided
Cleared yield — solved days	Historical median ~$11k / EFT hr	Mean ~$12.7k on 10 solved days (~18% above median)
Solver coverage	N/A	31/31 days resolved: 16 DNQ, 10 solved, 5 floor
Conversion elasticity	No formal demand model	Exponential decay fit, $R^2 \approx 0.97$ on 13 yield bins
SMX forecast	Seasonal mean baseline	Prophet, ~18% improvement vs baseline
Pricing team adoption	0% data-driven	Actively used as operational reference

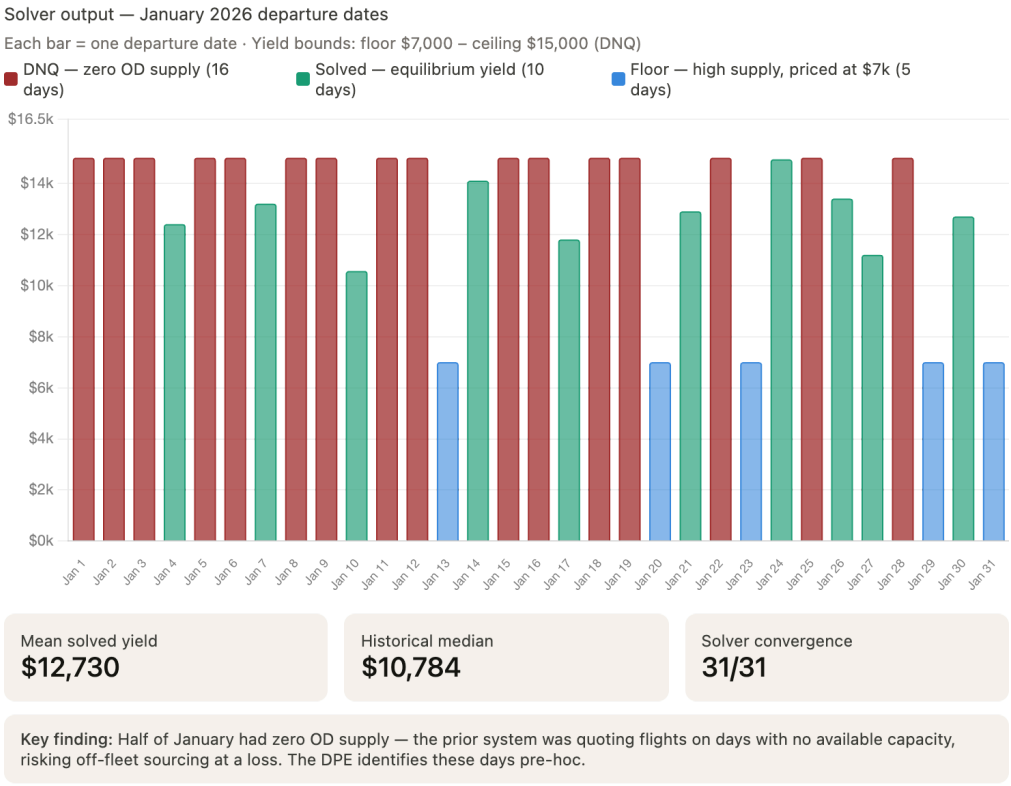


Figure 3 — Solver output, January 2026

---

5 — Key Learnings

- **Data freshness was a core product constraint, not just an infrastructure detail.**
  The engine was only as useful as the latest booking and fleet data available to it. In some cases, the business knew that new bookings had come in, but the database updates lagged by several hours. That meant the model could be technically correct on the data it had, while still being stale for real pricing decisions. Refresh latency directly affects trust, accuracy, and adoption.
- **Business validation was as important as model validation.**
  For two months, I reviewed the recommendations with the pricing team daily. That process helped me understand how they interpreted the outputs, when they trusted or overrode the model, and what operational constraints were missing from the data — for example cabotage, ferry legs, fleet positioning, and other real-world constraints. This feedback loop was critical to making the system usable.
- **Interpretability made adoption possible.**
  The equilibrium framing gave stakeholders a way to reason about the recommendation. They could see whether a price was driven by low supply, high demand, conversion sensitivity, or a guardrail. A black-box model with the same output likely would have been much harder to operationalize.
- **What I would do differently:**
  I would capture pricing overrides as structured feedback from day one. Every time the pricing team disagreed with the model, that was a labeled example explaining what the model was missing. I would also move earlier from raw quote counts toward more intent-qualified demand signals from the Salesforce opportunity funnel.
